QA — Training crash recovery

Contents

QA — Training crash recovery (A3a.9)

Re-runnable test for the orphan-session resumer. Proves that a training run which is killed mid-flight resumes from its latest checkpoint instead of being lost.

When to run

- After any change that touches `mlService._runTraining`, the checkpoint write block, `resumeOrphanedSessions`, or the `TrainingCheckpoint` schema.
- Before promoting an A3-track build to prod.
- Periodically against staging as a regression watch.

What it covers

- A checkpoint actually lands at the configured cadence.
- The orphan resumer picks up `RUNNING` sessions on next boot.
- Resumed sessions reach `COMPLETED` and unlock the model.

It does **not** cover the "no checkpoint → mark FAILED" branch (that's timing-dependent — verify by reading `mlService.resumeOrphans.test.js`).

Run book

The harness is two commands. The split is forced: the runner is inside the backend process being restarted, so it can't observe its own death.

Local (Docker Compose)

```
# 1. Start the session and wait for first checkpoint (~1–2 min for Quick qlearning)
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js training-recovery start

# Note the session id printed on the last stdout line. The command also prints
# the exact next commands to run, customised for this environment.

# 2. Trigger the "crash"
```

```
docker compose restart backend

# 3. Verify (≤5 min)
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js training-recovery verify --session <session-id>
```

Staging

```
# 1. Start (uses staging DB via --env staging; assumes flyctl proxy is up)
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js --env staging training-recovery start

# 2. Restart the backend
fly apps restart xo-backend-staging

# 3. Wait ~30s for the new instance to come up, then verify
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js --env staging training-recovery verify --session <session-id>
```

Expected PASS output

```
[ training-recovery ] env=staging session=sess_abc123
Polling for resume → completion (timeout 300s)…
  status=RUNNING checkpoint=1000
  status=RUNNING checkpoint=2000
  status=RUNNING checkpoint=3000
  status=RUNNING checkpoint=4000
  status=COMPLETED checkpoint=5000
✓ Recovery PASSED
  • Resume picked up from checkpoint at episode 1000
  • Final checkpoint at episode 5000 (target 5000)
  • BotSkill.status returned to: IDLE (expected IDLE)
```

Common failure modes

Symptom	Likely cause
`start` times out waiting for first checkpoint	`CHECKPOINT_GAP` raised or checkpoint write path broken. Tail backend logs for `TrainingCheckpoint write failed`.
`verify` stays in `PENDING` forever	Orphan resumer didn't pick up the session. Check that `resumeOrphanedSessions` is being called at boot (`grep -A2 "resumeOrphanedSessions" backend/src/index.js`) and that the filter (`status: 'RUNNING', pausedAt: null`) matches the session's actual state.
`verify` exits `FAILED` immediately	Session has no checkpoint — the start/restart sequence was too fast. Re-run with the start step given more time.
PASS but `BotSkill.status=TRAINING` anomaly	`_runTraining`'s `_finishSession` didn't fire on resume. Look at the per-algorithm finish path.
PASS but `completed without finishing iterations` anomaly	Resume ran but the loop's `startEpisode` handling regressed — episode index didn't advance.

Side effects

- Creates (or reuses) a `qa-recovery-seed` user and a `QA Recovery Skill` BotSkill in the target environment. These are dedicated test fixtures and do not appear in the leaderboard.
- Cancels any stale `RUNNING`/`PENDING` sessions on the test skill before starting, so a failed previous run won't block the next one.

Don’t run this in prod

The command refuses to run in prod via the standard `um` guardrails (`guardProduction()`); `--env prod` is not in the accepted list. A real prod recovery test should be done in pre-prod or on the staging mirror.